
AETERNUM CORE — INTERNAL SMART CONTRACT AUDIT REPORT

Project: Aeternum-core
Author: Internal Review
Date: 4th May 2026
Language: Solidity ^0.8.24

1. Executive Summary

This report presents an internal security review of the **Aeternum-core** smart contract, a non-custodial vault system with automated fund recovery via Chainlink Automation.

The contract demonstrates a **strong understanding of Solidity security patterns**, including:

- Checks-Effects-Interactions (CEI)
- Reentrancy protection
- Defensive validation
- Failure isolation in batch processing

Overall, the system is **well-architected and production-leaning**, but several **edge cases, economic risks, and operational considerations** remain.

2. Scope

Contracts Reviewed

- `AeternumVault.sol`

Dependencies

- OpenZeppelin `ReentrancyGuard`
 - Chainlink `AutomationCompatibleInterface`
-

3. Methodology

The review was conducted using:

- Manual code analysis
- Threat modeling
- Edge case simulation
- Known vulnerability pattern matching

Focus areas:

- Fund safety
 - State consistency
 - Reentrancy & external calls
 - Automation reliability
 - Economic correctness
-

4. System Overview

AeternumVault allows users to:

- Lock ETH into a vault
- Define inactivity conditions
- Automatically transfer funds to a backup address

Recovery is triggered when:

```
block.timestamp >= lastActivity + inactivityPeriod
```

Execution is handled via:

- `checkUpkeep()` (off-chain scanning)
 - `performUpkeep()` (on-chain execution)
-

5. Severity Classification

Level	Description
Critical	Direct loss of funds or permanent lock
High	Serious vulnerability or exploit path
Medium	Unexpected behavior or edge-case risk

Level	Description
Low	Minor inefficiency or best-practice issue
Informational Observations / improvements	

6. Findings

6.1 [HIGH] Unbounded Registry Growth → Automation Scalability Risk

Description

`s_registeredWallets` grows indefinitely with no pruning beyond user cancellation or recovery.

`checkUpkeep()` iterates over this array (even if paginated), creating:

- Increasing off-chain computation cost
- Operational complexity for Chainlink upkeep configuration

Impact

- Automation may become inefficient or unreliable at scale
- Missed recovery triggers if pagination is misconfigured

Recommendation

- Introduce:
 - Sharded registries OR
 - Multiple upkeep registrations enforced at protocol level
 - Consider emitting indexing events for off-chain indexing instead of relying on full scans
-

6.2 [MEDIUM] Recovery Failure Retry Design Can Be Griefed

Description

If a backup address consistently reverts:

- System retries up to 3 times
- Re-adds wallet to registry each time

Impact

- Repeated gas consumption
- Potential griefing vector for automation nodes

- Temporary registry bloat

Mitigation Present

- Retry cap (`MAX_RECOVERY_ATTEMPTS = 3`)
- Abandonment mechanism

Recommendation

- Consider:
 - Exponential backoff strategy
 - Delay between retries
-

6.4 [MEDIUM] No Explicit Gas Optimization for Large Batches

Description

`performUpkeep()` processes up to 50 wallets sequentially.

Worst-case:

- Multiple failing transfers
- High gas usage per transaction

Impact

- Risk of hitting block gas limits in extreme conditions

Recommendation

- Consider dynamic batch sizing based on gas left
 - Or partial execution with checkpointing
-

6.5 [LOW] Direct ETH Transfer Reverts (UX Risk)

Description

```
receive() external payable {  
    revert AeternumVault__DirectTransferNotAllowed();  
}
```

Impact

- Users sending ETH directly will lose gas and fail
- No recovery mechanism

Recommendation

- Optionally emit event before revert for better UX debugging
-

6.6 [LOW] Missing Two-Step Treasury Transfer

Description

Treasury can be changed instantly via:

```
updateTreasury(address newTreasury)
```

Impact

- Risk of accidental misconfiguration

Recommendation

- Implement:
 - `proposeTreasury()`
 - `acceptTreasury()`
-

6.7 [INFORMATIONAL] Strong CEI + Reentrancy Design

The contract correctly:

- Updates state before external calls
- Uses `nonReentrant` on sensitive functions
- Handles failure recovery safely

This is **well implemented and production-grade**.

6.8 [INFORMATIONAL] Clean Swap-and-Pop Registry Management

Efficient $O(1)$ removal:

- Prevents index corruption
- Safe under batch operations

6.9 [INFORMATIONAL] Robust Recovery Re-validation

Each recovery execution re-checks:

- Active status
- Balance
- Time condition

Prevents:

- Stale automation data exploits
 - Race conditions
-

7. Attack Surface Analysis

External Calls

- `call{value: amount}` used in:
 - `withdraw`
 - `send`
 - `recovery`

Mitigations:

- CEI pattern
 - ReentrancyGuard
-

Automation Layer Risks

- Dependence on Chainlink nodes
- Misconfigured upkeep ranges

Mitigation:

- Pagination design
 - Batch size limits
-

Economic Risks

- Fee mispayment
 - No refund logic
 - Premium subscription simplicity
-

8. Gas & Efficiency Review

Efficient

- O(1) removal (swap-and-pop)
- Packed struct usage
- Minimal storage writes

Potential Improvements

- Reduce redundant storage reads in loops
 - Cache frequently used values
-

9. Design Strengths

- Strong separation of concerns
 - No admin control over user funds
 - Failure isolation in batch execution
 - Deterministic and predictable state transitions
 - Clean and readable codebase
-

10. Final Assessment

Security Rating: HIGH (Production Candidate with Improvements)

The contract is:

- Secure by design
- Well thought-out
- Architecturally sound

However, before mainnet deployment:

- Economic logic should be tightened
 - Automation scalability should be addressed
 - Minor UX and governance improvements added
-

11. Recommended Next Steps

Before production:

1. Add treasury transfer safety
 2. Stress test with large registry sizes
 3. Simulate Chainlink upkeep behavior at scale
 4. Consider external audit
-

12. Conclusion

AeternumVault demonstrates **advanced smart contract engineering maturity**, particularly in:

- Failure handling
- Automation integration
- Security pattern usage

With targeted refinements, it is a strong candidate for:

- Mainnet deployment
 - External audit
 - Production adoption
-